

Internet of Things

Lecture 1: Data Propagation Algorithms

Prof. Constantinos Marios Angelopoulos

mangelopoulos@ihu.gr

MSc in Web Intelligence

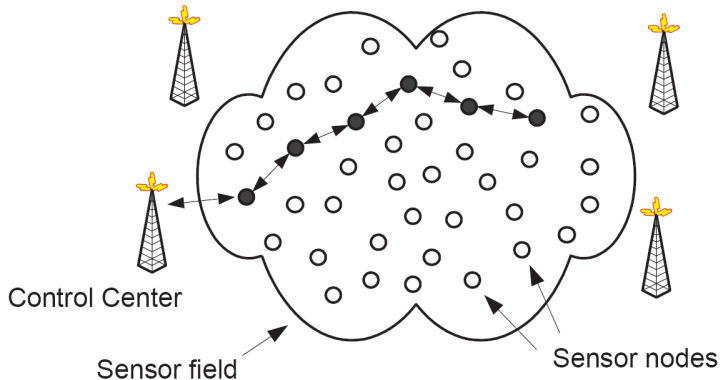


INTERNATIONAL
HELLENIC
UNIVERSITY



The problem

“How can sensor p , via cooperation with the rest of the sensors in the network, propagate information about event $\mathcal{E}$ to the control center(s)?”





Lecture Overview

- A Detailed presentation of Directed Diffusion
- B Detailed presentation of LEACH



Flat vs Hierarchical Routing

- **Flat Routing:** All nodes in the network have similar role regarding the routing of data. No special nodes are used.
 - Example: Directed Diffusion
- **Hierarchical Routing:** Special nodes assume greater responsibility regarding routing of data than most nodes inside the network. Super nodes – cluster heads.
 - Example: LEACH



A. Directed Diffusion

C. Intanagonwiwat, R. Govindan, D. Estrin

**“Directed Diffusion: A Scalable and Robust Communication
Paradigm for Sensor Networks”**

6th Annual International Conference on Mobile Computing and Networking, 2000



Introduction

Sensor networks require coordination in order to perform distributed event sensing.

Directed diffusion is a

- data-centric
- application aware
- energy efficient

coordination paradigm for delivering sensed data.



Introduction

Challenges for directed diffusion:

- **Scalability:** The protocol should maintain correctness when the network scales a)in diameter; b)in number of nodes; c)in data volume
- **Energy efficiency:** The protocol should be kind in energy consumption so as to guarantee good network lifetime
- **Robustness / Fault tolerance:** The protocol should cope well with dynamics (at least to some degree)
- **Efficient routing:** In terms of a) number of nodes involved in propagating a single message; b)time delay; c)successful delivery rate.



Directed Diffusion

Directed diffusion elements:

- Interest messages
- Data messages
- Gradients
- Reinforcements of gradients



Interests and tasks

- An interest contains the description of a sensing task.
- Task descriptions are identified e.g. by a list of attribute-value pairs.
- The description specifies an interest for data matching the attributes.

Example of an interest:

```
type = wheeled vehicle      // detect vehicle location
interval = 100 ms           // send events every 100 ms
duration = 10 seconds       // for the next 10 seconds
rect = [-100, 100, 200, 400] // from nodes within rectangle
```



Interests - What happens at the Sink

Interests are injected into the network at some (possibly arbitrary) node, the **sink**. The sink **diffuses** the interests through the sensor network.

- For each interest a task is generated.
- For each active task the sink generates an **exploratory** interest message.

```
type = wheeled vehicle
interval = 0.1s
rect = [-100, 100, 200, 400]
timestamp = 01:20:40
expiresAt = 01:30:40
```

Note that:

- An interest is periodically **refreshed** by the sink.
- Interests **do not contain information about the sink**.
- Interests can be **aggregated** e.g. interests with identical types and completely overlapping rect attributes.



Interests - What happens at the Nodes

Every node maintains an interest cache.

Upon interest reception a new entry is created in the cache.

This includes:

- A **timestamp** that stores the timestamp of the last received matching interest.
- A **gradient** entry, up to one per neighbor, is created. Each gradient stores:
 - Data rate.
 - Duration.

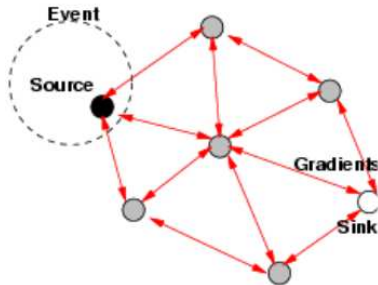
Note that:

- If an interest already exists in cache only a new gradient is created for this interest.
- An interest is erased from cache only when every gradient has expired.



Gradients

Gradients are formed by **local interaction** of neighboring nodes.
Neighboring nodes establish a gradient towards each other.
Gradients store a **value** and a direction.
Gradients facilitate "pulling down" data towards the sink.



Gradient establishment when flooding an interest.



Data propagation

A sensor that receives an interest it can serve, begins sensing.

As soon as a matching event is detected:

- The node computes the highest requested event rate among it's gradients.
- The node generates event samples at this rate.

```
type = wheeled vehicle // type of vehicle
instance = truck // instance of this type
location = [125, 220] // node location
intensity = 0.6 // signal amplitude measure
confidence = 0.85 // confidence in the match
timestamp = 01:20:40 // local event generation time
```

- Data messages are unicasted individually to the relevant neighbors (neighbors where gradients point to).



Data propagation

Every node maintains a data cache.

A node receives a data message.

- If the data message doesn't have a matching interest or data exists in cache, the message is dropped.
- Otherwise, the message is added to the data cache and is forwarded.

To forward a message:

- A node determines the data rate of received events by examining it's data cache.
- If the requested data rate on all gradients is greater or equal to the rate of incoming events, the message is forwarded.
- If some gradients have lower data rate then the data is **down-converted** to the appropriate rate.



Reinforcement for Path Establishment and Truncation

- 1 The sink initially repeatedly diffuses an interest for a low-rate event notification, the generated messages are called exploratory messages.
- 2 The gradients created by exploratory messages are called exploratory gradients and have low data rate.
- 3 As soon as a matching event is detected, exploratory events are generated and routed back to the sink.
- 4 After the sink receives those exploratory events, it **reinforces** one particular neighbor in order to “draw down” real data.
- 5 The gradients setup for receiving high quality tracking events are called **data** gradients.



Path Establishment Using Positive Reinforcement

To reinforce a neighbor, the sink re-sends the original interest message with a smaller interval (higher rate).

```
type = wheeled vehicle
interval = 10ms
rect = [-100, 100, 200, 400]
timestamp = 01:22:35
expiresAt = 01:30:40
```

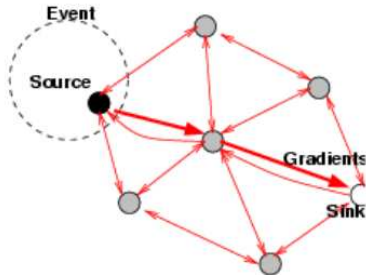
Upon reception of this message a node updates the corresponding gradient to match the requested data rate.

If the requested data rate is higher than the rate of incoming events, the node reinforces one of it's neighbors.



Path Establishment Using Positive Reinforcement

The selection of a neighbour for reinforcement is based on local criteria
e.g. the neighbour that reported first a new event is reinforced.
The data cache is used to determine which criteria are fulfilled.



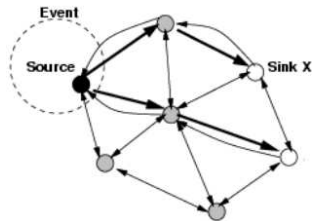
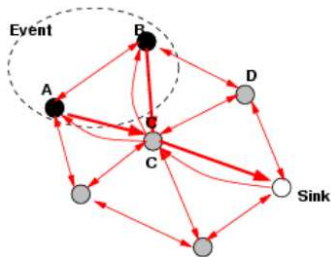
Gradient reinforcement.



Reinforcement for Path Establishment and Truncation

The above scheme is reactive to changes; whenever one path delivers an event faster than others, it is reinforced.

This scheme supports the existence of multiple sinks and multiple sources in the network.

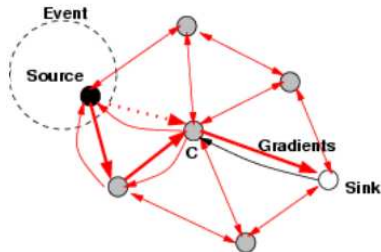




Local Repair for Failed Paths

Paths may degrade over time.

A node can detect path degradation i.e. by noticing reduced event rates. Intermediate nodes on a path can apply the reinforcement rules and repair the path.





Negative Reinforcement

A mechanism for truncating paths is required.

- Gradients timeout unless they are explicitly reinforced.
- Negative reinforcement of a path.

Negative reinforcement is achieved by sending an interest with exploratory data rate.

If all outgoing gradients of a node are exploratory the node negatively reinforces it's neighbours.

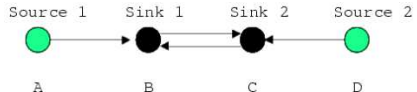
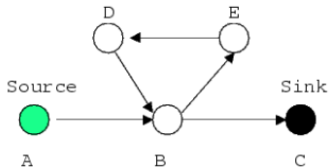
Negative reinforcement is applied when certain criteria are met
e.g. a gradient doesn't deliver any new messages for an amount of time.



Loop removal

Loops don't deliver new events.

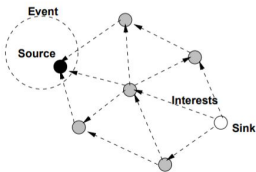
Some loops can't be broken.



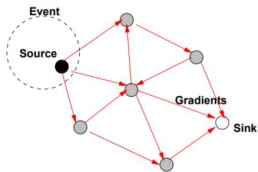
Removable and unremovable loops.



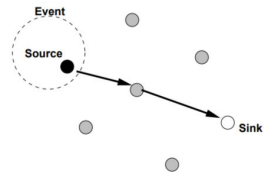
Direct Diffusion - Overview



(a) Interest propagation



(b) Initial gradients set up



(c) Data delivery along re-inforced path



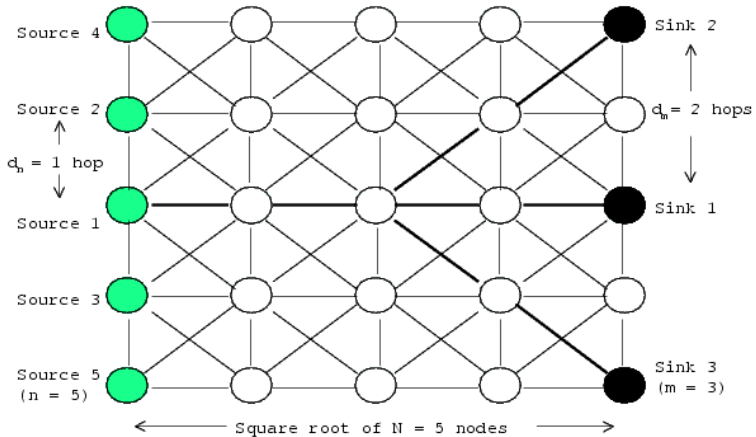
Analytic Evaluation

Let n sources and m sinks in a square grid topology of size $\sqrt{N} \times \sqrt{N}$. Let d_n (d_m) the number of hops between two adjacent sources (sinks). Measure the total cost of transmission and reception of one event from each source to all sinks.

- Flooding
- Omniscient Multicast
 - Uses minimum height multicast trees
- Directed diffusion



Topology



Example of a square grid topology



Flooding

Every event is broadcasted to all nodes in the network.

Each node broadcasts an event only once, nN transmissions.

On each channel each message is sent twice,
 $2n(2\sqrt{N}(\sqrt{N} - 1) + 2(\sqrt{N} - 1)^2)$ receptions.

$$C_f = nN + 4n(\sqrt{N} - 1)(2\sqrt{N} - 1)$$

The order of C_f is $O(nN)$.



Omniscient Multicast

Each source transmits events along a shortest-path multicast tree. Let T_i the tree with root source i , let $C(T_i)$ the data delivery cost for tree T_i .

Trees overlap but each message is transmitted once and received once on each tree edge. The cost of messages transmitted on a tree can be expressed as the cost of the tree T_1

It is proven that $C(T_i)$ is in the order of $O(\sqrt{N})$ for $m \ll \sqrt{N}$. The cost of omniscient multicast C_o is in the order of $O(n\sqrt{N})$



Directed diffusion

Assume that established paths match the shortest-path multicast trees.

If all sources send identical location estimates diffusion can perform application level duplicate suppression.

Trees overlap but each message is transmitted only once and received only once on each tree edge.

Similar to C_o , C_d is in the order of $O(n\sqrt{N})$ for $m \ll \sqrt{N}$.



Simulation

Flooding, Omniscient Multicast and Directed Diffusion were simulated on **ns-2**.

Metrics:

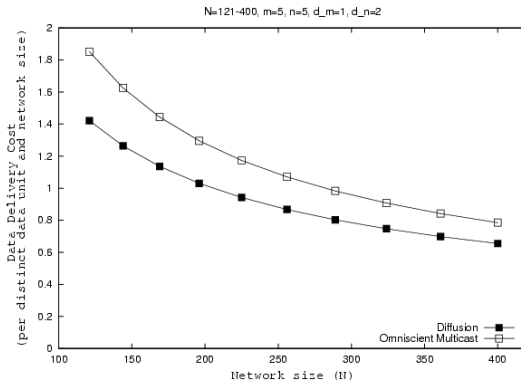
- Average dissipated energy.
- Average delay.
- Distinct-event delivery ratio.



Comparison - Data Delivery Cost

C_f is several orders of magnitude higher than C_o and C_d

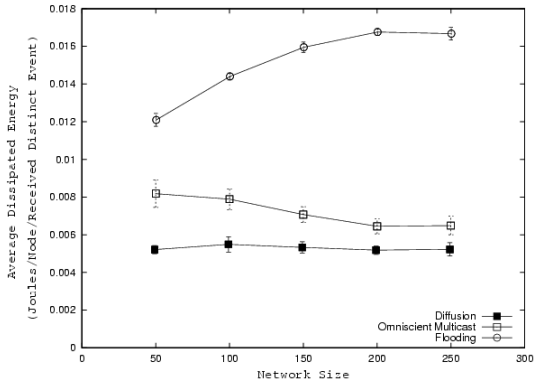
Although C_o and C_d are in the same order of magnitude, $C_o > C_d$



Impact of network size



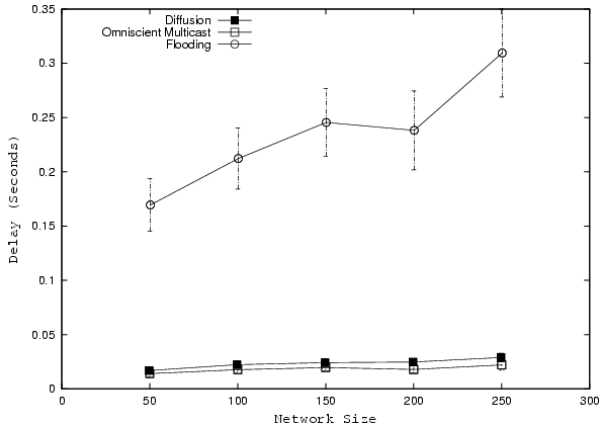
Average dissipated energy



- Omniscient multicast dissipates significant less energy than flooding since events are delivered along a single path.
- Directed diffusion outperforms omniscient multicast as **in-network aggregation** suppresses duplicate messages.



Average delay



- Flooding is remarkably slower due to collisions in the MAC layer.
- Directed diffusion performs comparably to omniscient multicast.



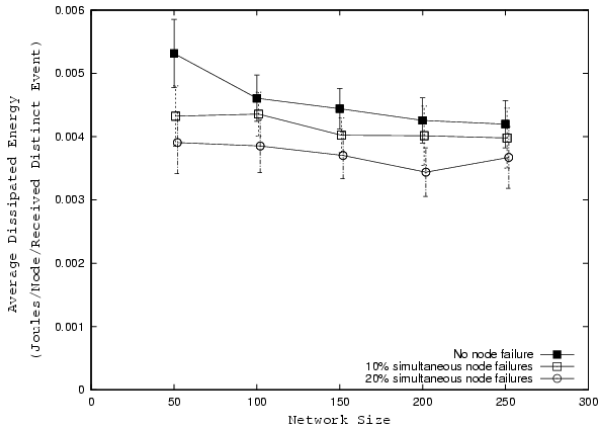
Impact of dynamics on DD

Assumptions:

- Node failures occur randomly in the network.
- Half of node failures occur on nodes on the shortest path trees.
- Sources send different location estimates.



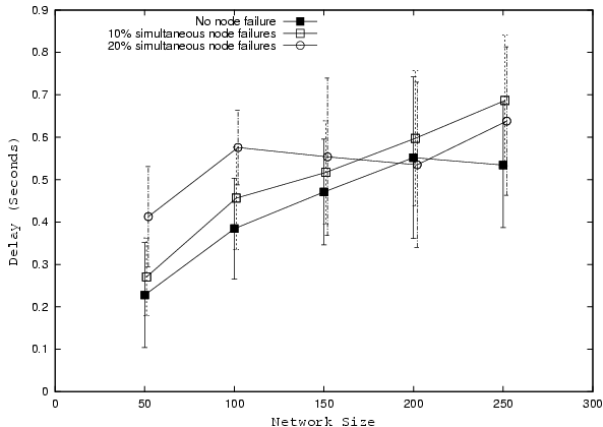
Average dissipated energy



- Average dissipated energy doesn't increase significantly.
- Instead an improvement is observed in some cases.
- Negative reinforcement rules allow a number of high quality paths.



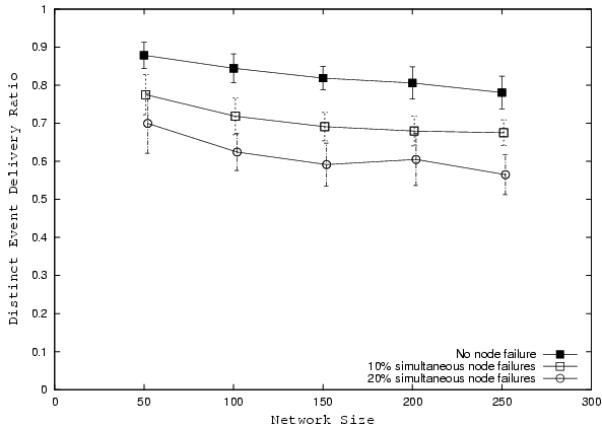
Average delay



- Average delay increases but no more than 20%.



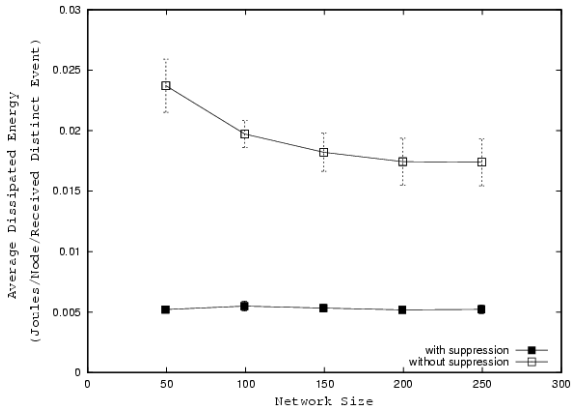
Event delivery ratio



- Event delivery ratio reduces proportionally to node failure percentage.



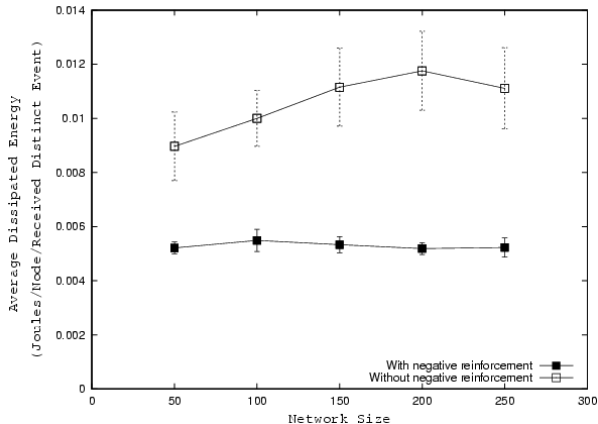
Impact of Data Aggregation



- Without aggregation, diffusion dissipates 3 (for larger networks) to 5 times (small network sizes) more energy.
- Longer (higher latency) alternative paths form in large networks, which are pruned by negative reinforcement.



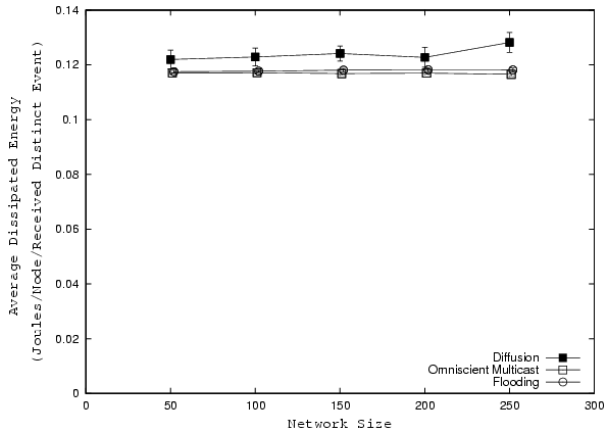
Impact of Negative reinforcement



- Without negative reinforcement 2 times more energy is dissipated.



Impact of Radio Model



- Radios that consume high energy amounts when in the idle state, affect diffusion.
- Idle time dominates the performance of all schemes.



Conclusions about Directed Diffusion

- Directed diffusion has the potential for significant energy efficiency.
- Diffusion mechanisms are stable under the range of network dynamics considered.
- The sensor radio MAC layer design affects directed diffusion significantly.



B. LEACH (Low Energy Adaptive Clustering Hierarchy)

W.R. Heinzelman, A. Chandrakasan & H. Balakrishnan
**“Energy-Efficient Communication Protocol for
Wireless Microsensor Networks”**

33rd Hawaii International Conference on System Sciences, HICCS-2000



Main features of LEACH

What is LEACH?

- *cluster-based* protocol that *minimizes* energy dissipation in sensor network.
- key features:
 - **localized coordination** and control for cluster set-up and orientation.
 - **randomized rotation** of the cluster “base station” or “cluster heads” and the corresponding clusters.
 - **local compression** to reduce global communication.



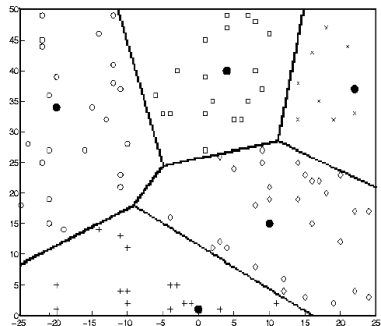
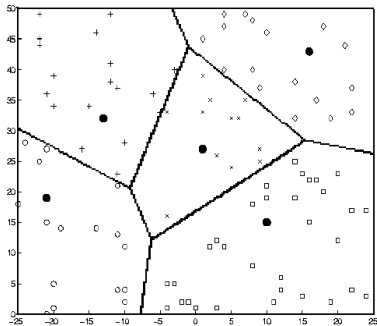
Intuitive description of LEACH

How does LEACH work:

- Network is partitioned in *clusters*.
- Each cluster has one *cluster-head*.
- Each non *cluster-node* sends data to the head of the cluster it belongs.
- Cluster-heads gather the sent data, compress them and send them to the base-station *directly*.



Dynamic Clusters





Operation of LEACH

- LEACH operates in *rounds*.
- Each round in LEACH consists of *phases*.
 - Advertisement Phase.
 - Cluster Set-Up Phase.
 - Steady Phase.



Advertisement Phase (1/3)

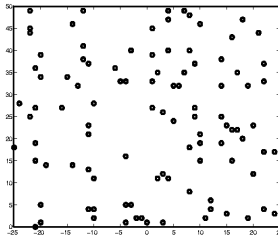
Election: Node n decides with probability $T(n)$ to elect itself *cluster-head*

$$T(n) = \begin{cases} \frac{P}{1 - P * (r \bmod \frac{1}{P})} & \text{if } n \in G \\ 0 & \text{otherwise} \end{cases}$$

P : the desired percentage of cluster heads.

r : the current round.

G : the set of nodes that have not been cluster-heads in the last $\frac{1}{P}$ rounds.





Example

Assume that $n = 100$ and $P = 20\% \Rightarrow$ on the average, the network will have 20 cluster heads.

- $r=0; G=100 \mapsto T(n) = \frac{0.2}{1-0.2(0 \bmod \frac{1}{0.2})} = 0.2 \Rightarrow 0.2 \times 100 = 20$ CHs
- $r=1; G=80 \mapsto T(n) = \frac{0.2}{1-0.2(1 \bmod \frac{1}{0.2})} = \frac{1}{4} \Rightarrow 0.25 \times 80 = 20$ CHs
- $r=2; G=60 \mapsto T(n) = \frac{0.2}{1-0.2(2 \bmod \frac{1}{0.2})} = \frac{1}{3} \Rightarrow \frac{1}{3} \times 60 = 20$ CHs
- $r=3; G=40 \mapsto T(n) = \frac{0.2}{1-0.2(3 \bmod \frac{1}{0.2})} = \frac{1}{2} \Rightarrow 0.5 \times 40 = 20$ CHs
- $r=4; G=20 \mapsto T(n) = \frac{0.2}{1-0.2(4 \bmod \frac{1}{0.2})} = 1 \Rightarrow 1 \times 20 = 20$ CHs

What happens when $r=5$?



Example

Assume that $n = 100$ and $P = 20\% \Rightarrow$ on the average, the network will have 20 cluster heads.

- $r=0; G=100 \mapsto T(n) = \frac{0.2}{1-0.2(0 \bmod \frac{1}{0.2})} = 0.2 \Rightarrow 0.2 \times 100 = 20$
CHs
- $r=1; G=80 \mapsto T(n) = \frac{0.2}{1-0.2(1 \bmod \frac{1}{0.2})} = \frac{1}{4} \Rightarrow 0.25 \times 80 = 20$ CHs
- $r=2; G=60 \mapsto T(n) = \frac{0.2}{1-0.2(2 \bmod \frac{1}{0.2})} = \frac{1}{3} \Rightarrow \frac{1}{3} \times 60 = 20$ CHs
- $r=3; G=40 \mapsto T(n) = \frac{0.2}{1-0.2(3 \bmod \frac{1}{0.2})} = \frac{1}{2} \Rightarrow 0.5 \times 40 = 20$ CHs
- $r=4; G=20 \mapsto T(n) = \frac{0.2}{1-0.2(4 \bmod \frac{1}{0.2})} = 1 \Rightarrow 1 \times 20 = 20$ CHs

What happens when $r=5$?



Example

Assume that $n = 100$ and $P = 20\% \Rightarrow$ on the average, the network will have 20 cluster heads.

- $r=0; G=100 \mapsto T(n) = \frac{0.2}{1-0.2(0 \bmod \frac{1}{0.2})} = 0.2 \Rightarrow 0.2 \times 100 = 20$ CHs
- $r=1; G=80 \mapsto T(n) = \frac{0.2}{1-0.2(1 \bmod \frac{1}{0.2})} = \frac{1}{4} \Rightarrow 0.25 \times 80 = 20$ CHs
- $r=2; G=60 \mapsto T(n) = \frac{0.2}{1-0.2(2 \bmod \frac{1}{0.2})} = \frac{1}{3} \Rightarrow \frac{1}{3} \times 60 = 20$ CHs
- $r=3; G=40 \mapsto T(n) = \frac{0.2}{1-0.2(3 \bmod \frac{1}{0.2})} = \frac{1}{2} \Rightarrow 0.5 \times 40 = 20$ CHs
- $r=4; G=20 \mapsto T(n) = \frac{0.2}{1-0.2(4 \bmod \frac{1}{0.2})} = 1 \Rightarrow 1 \times 20 = 20$ CHs

What happens when $r=5$?



Example

Assume that $n = 100$ and $P = 20\% \Rightarrow$ on the average, the network will have 20 cluster heads.

- $r=0; G=100 \mapsto T(n) = \frac{0.2}{1-0.2(0 \bmod \frac{1}{0.2})} = 0.2 \Rightarrow 0.2 \times 100 = 20$ CHs
- $r=1; G=80 \mapsto T(n) = \frac{0.2}{1-0.2(1 \bmod \frac{1}{0.2})} = \frac{1}{4} \Rightarrow 0.25 \times 80 = 20$ CHs
- $r=2; G=60 \mapsto T(n) = \frac{0.2}{1-0.2(2 \bmod \frac{1}{0.2})} = \frac{1}{3} \Rightarrow \frac{1}{3} \times 60 = 20$ CHs
- $r=3; G=40 \mapsto T(n) = \frac{0.2}{1-0.2(3 \bmod \frac{1}{0.2})} = \frac{1}{2} \Rightarrow 0.5 \times 40 = 20$ CHs
- $r=4; G=20 \mapsto T(n) = \frac{0.2}{1-0.2(4 \bmod \frac{1}{0.2})} = 1 \Rightarrow 1 \times 20 = 20$ CHs

What happens when $r=5$?



Example

Assume that $n = 100$ and $P = 20\% \Rightarrow$ on the average, the network will have 20 cluster heads.

- $r=0; G=100 \mapsto T(n) = \frac{0.2}{1-0.2(0 \bmod \frac{1}{0.2})} = 0.2 \Rightarrow 0.2 \times 100 = 20$ CHs
- $r=1; G=80 \mapsto T(n) = \frac{0.2}{1-0.2(1 \bmod \frac{1}{0.2})} = \frac{1}{4} \Rightarrow 0.25 \times 80 = 20$ CHs
- $r=2; G=60 \mapsto T(n) = \frac{0.2}{1-0.2(2 \bmod \frac{1}{0.2})} = \frac{1}{3} \Rightarrow \frac{1}{3} \times 60 = 20$ CHs
- $r=3; G=40 \mapsto T(n) = \frac{0.2}{1-0.2(3 \bmod \frac{1}{0.2})} = \frac{1}{2} \Rightarrow 0.5 \times 40 = 20$ CHs
- $r=4; G=20 \mapsto T(n) = \frac{0.2}{1-0.2(4 \bmod \frac{1}{0.2})} = 1 \Rightarrow 1 \times 20 = 20$ CHs

What happens when $r=5$?



Example

Assume that $n = 100$ and $P = 20\% \Rightarrow$ on the average, the network will have 20 cluster heads.

- $r=0; G=100 \mapsto T(n) = \frac{0.2}{1-0.2(0 \bmod \frac{1}{0.2})} = 0.2 \Rightarrow 0.2 \times 100 = 20$ CHs
- $r=1; G=80 \mapsto T(n) = \frac{0.2}{1-0.2(1 \bmod \frac{1}{0.2})} = \frac{1}{4} \Rightarrow 0.25 \times 80 = 20$ CHs
- $r=2; G=60 \mapsto T(n) = \frac{0.2}{1-0.2(2 \bmod \frac{1}{0.2})} = \frac{1}{3} \Rightarrow \frac{1}{3} \times 60 = 20$ CHs
- $r=3; G=40 \mapsto T(n) = \frac{0.2}{1-0.2(3 \bmod \frac{1}{0.2})} = \frac{1}{2} \Rightarrow 0.5 \times 40 = 20$ CHs
- $r=4; G=20 \mapsto T(n) = \frac{0.2}{1-0.2(4 \bmod \frac{1}{0.2})} = 1 \Rightarrow 1 \times 20 = 20$ CHs

What happens when $r=5$?



Example

Assume that $n = 100$ and $P = 20\% \Rightarrow$ on the average, the network will have 20 cluster heads.

- $r=0; G=100 \mapsto T(n) = \frac{0.2}{1-0.2(0 \bmod \frac{1}{0.2})} = 0.2 \Rightarrow 0.2 \times 100 = 20$ CHs
- $r=1; G=80 \mapsto T(n) = \frac{0.2}{1-0.2(1 \bmod \frac{1}{0.2})} = \frac{1}{4} \Rightarrow 0.25 \times 80 = 20$ CHs
- $r=2; G=60 \mapsto T(n) = \frac{0.2}{1-0.2(2 \bmod \frac{1}{0.2})} = \frac{1}{3} \Rightarrow \frac{1}{3} \times 60 = 20$ CHs
- $r=3; G=40 \mapsto T(n) = \frac{0.2}{1-0.2(3 \bmod \frac{1}{0.2})} = \frac{1}{2} \Rightarrow 0.5 \times 40 = 20$ CHs
- $r=4; G=20 \mapsto T(n) = \frac{0.2}{1-0.2(4 \bmod \frac{1}{0.2})} = 1 \Rightarrow 1 \times 20 = 20$ CHs

What happens when $r=5$?



Advertisement Phase (2/3)

On every round we want on the average the same number of cluster-heads in the network, i.e. $N \cdot P$ (N : number of nodes).

On first round we have NP cluster-heads.

On second round we have $N(1 - P)P_1$ cluster-heads.

$$N(1 - P)P_1 = N \cdot P \Rightarrow P_1 = \frac{P}{1 - P} = T(n)$$

For third round we have $N(1 - P)(1 - P_1)P_2$ cluster-heads.

$$N(1 - P)(1 - P_1)P_2 = N \cdot P \Rightarrow$$
$$P_2 = \frac{P}{1 + P \cdot P_1 - (P + P_1)} = \frac{1}{1 - 2P} = T(n)$$

By induction, we get $T(n)$



Advertisement Phase (3/3)

Cluster-Head-Advertisement:

- Each cluster-head *broadcasts* an advertisement message to the rest of the nodes using CSMA-MAC protocol.
- Non cluster-head nodes hear the advertisements of all cluster-head nodes.
- Each non-cluster head node decides which cluster head to join on some local criteria; e.g. by choosing the one that has the strongest signal.



Cluster Set-Up Phase

- Each cluster head is informed of the members of its cluster.
- The cluster head creates a *TDMA schedule*
- Cluster-head broadcasts the schedule back to the cluster members.



Steady Phase

- Non cluster-heads
 - *Sense* the environment.
 - *Send* their data to the cluster head during their transmission time.
- Cluster-heads
 - *Receive* data from non cluster-head nodes.
 - *Compress* the data they have received.
 - *Send* their data to the base station.

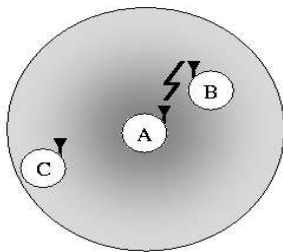
The Duration of steady phase is “a priori” determined.



Multiple Clusters

PROBLEM

Transmissions in one cluster can degrade communications in nearby clusters.



SOLUTION

Cluster-head chooses randomly from a list of spreading codes and informs all the members of its cluster.



Hierarchical Clustering Extensions

- Non cluster-heads communicate with their cluster-heads
- Cluster-heads communicate with *super-cluster-heads*
- And so on ...

Advantages: Saves a lot of energy for larger networks, more realistic

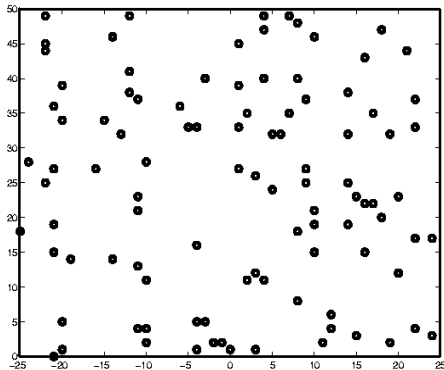
Disadvantages: More complicated implementation, latency



Experimental Evaluation of LEACH

We are given a network of *100 nodes*.

In the area:



The *base station* is placed at $(0, 100)$.



Experimental Evaluation of LEACH

There is a comparative study between:

- Minimum-Energy-Transmission.
- Direct Transmission.
- LEACH.

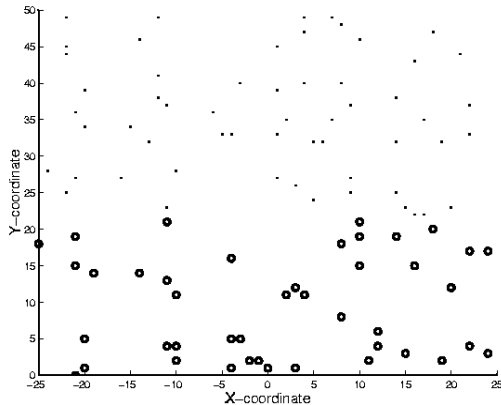
The evaluation measures:

- Dead Nodes' *Distribution*.
- Total *Energy Dissipation*.
- System *Life-time*.



Distribution of dead nodes (1/3)

Minimum-Energy-Transmission

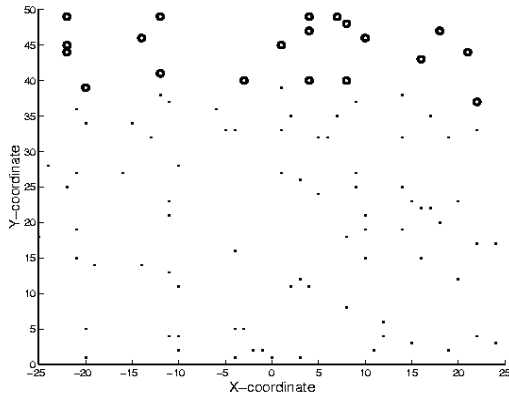


- After 180 rounds nodes *closer* to the sink die faster!



Distribution of dead nodes(2/3)

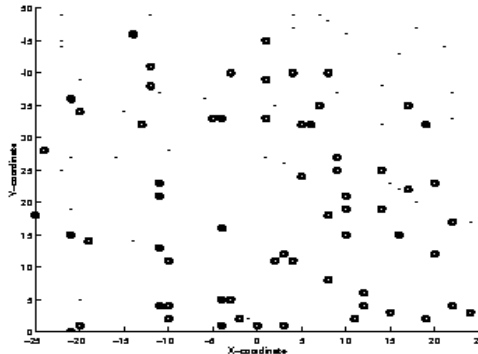
Direct-Transmission



- After 180 rounds nodes *further* to the sink die faster!



Distribution of dead nodes(3/3)

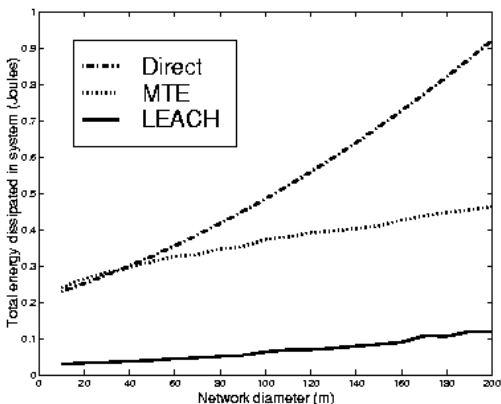


LEACH

- After 1200 rounds nodes die in *uniform* manner.



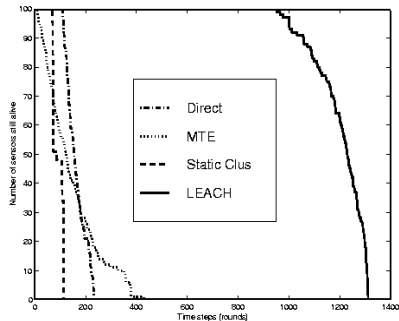
Total energy dissipation - network diameter



- *LEACH* reduces 7x to 8x compared to *Direct-Transmission*.
- *LEACH* reduces 4x to 8x compared to *Minimum-Energy-Transmission*.



System Lifetime (1/2)



- LEACH more than *doubles* the useful system lifetime.
- It takes *8-times* longer for the first node to die in LEACH.
- It takes *3-times* longer for the last node to die in LEACH.



System Lifetime (2/2)

Energy (J/node)	Protocol	Round first node dies	Round last node dies
0.25	Direct	55	117
	MTE	5	221
	Static Clustering	41	67
	LEACH	394	665
0.5	Direct	109	234
	MTE	8	429
	Static Clustering	80	110
	LEACH	932	1312
1	Direct	217	468
	MTE	15	843
	Static Clustering	106	240
	LEACH	1848	2608



Conclusions about LEACH

LEACH is a cluster-based routing protocol.

- *Minimizes* energy dissipation through compression techniques in cluster-level.
- *Distributes* the load to all the nodes at different times.

In experimental evaluation

- LEACH reduces communication energy as much as 8x compared to DT or MTE.
- In LEACH the first node death occurs over 8 times later compared DT or MTE.
- In LEACH the last node death occurs over 3 times later compared to DT or MTE.



Criticism – Variations & Improvements

- Unrealistic radio assumptions (cluster-heads must transmit directly to gateways)
- No real-life implementation
- No good for reactive applications (vs. proactive ones)
- TEEN, PEGASIS, examples of variations improving on latency and realism
- Other protocols use more hierarchical levels or/and multihop routing between cluster-heads.



Thank You!

Questions?